

CHAPTER 5 — POINTERS AND DATA TYPES

SLIDE 1 — WHAT ARE POINTERS AND DATA TYPES?

MEET POINTERS AND DATA TYPES!

A pointer stores the **address of another variable**.

But a pointer also has a **data type**.

The data type tells C **what type of value the pointer is pointing to**.

Simple Definition

A pointer's data type tells us the type of value stored at the address it points to.

LOOK AT THIS!

None

```
int x = 10;  
int *p = &x;
```

Here:

None

```
x → 10  
p → address of x
```

Since `x` is an `int`:

None

```
int *p;
```

is an **integer pointer**.

EASY ANALOGY

Think of a pointer like an address label .

The label also tells you **what kind of thing is at that address**.

None

 Address →  int value
 Address →  char value
 Address →  float value

★ REMEMBER!

Data type of pointer → Data type of value it points to

SLIDE 2 — POINTERS WITH DIFFERENT DATA TYPES

CAN POINTERS POINT TO DIFFERENT DATA TYPES?

Yes! 

Pointers can point to variables of different data types.

INTEGER

None

```
int x = 10;  
int *p = &x;
```

p points to an **integer**.

CHARACTER

None

```
char ch = 'A';  
char *p = &ch;
```

p points to a **character**.

FLOAT

None

```
float marks = 85.5;  
float *p = &marks;
```

p points to a **float**.

LOOK AT THIS!

None

```
int    → int *  
char   → char *  
float  → float *  
double → double *
```

MEMORY TRICK

Variable type → Pointer type

If the variable is:

None

```
int
```

the pointer is:

None

`int *`

★ TAKEAWAY

The pointer type should match the type of variable it points to.

🟡 SLIDE 3 — 🔍 ACCESSING VALUES WITH DIFFERENT POINTER TYPES

😞 HOW DO WE ACCESS THE VALUE?

We use the **dereference operator** `*`.

Example

None

```
int x = 25;
int *p = &x;
printf("%d", *p);
```

Output:

None

25

Because:

None

`p` → address of `x` 📌
`*p` → value of `x` 🔍

CHARACTER EXAMPLE

None

```
char ch = 'A';  
char *p = &ch;  
printf("%c", *p);
```

Output:

None

A

FLOAT EXAMPLE

None

```
float marks = 85.5;  
float *p = &marks;  
printf("%f", *p);
```

Output:

None

85.500000

EASY ANALOGY

Think of different pointers as different **labels** :

None

```
int *    →  Integer label  
char *   →  Character label  
float *  →  Decimal label
```

★ REMEMBER!

*p gives the value stored at the address.

SLIDE 4 — POINTER TYPE MUST MATCH

WHY IS THE DATA TYPE IMPORTANT?

Because C needs to know **what kind of data is stored at that address**.

For example:

None

```
int x = 50;  
int *p = &x;
```

This is correct because:

None

```
int variable → int pointer
```

LOOK AT THIS

None

```
char ch = 'A';  
char *p = &ch;
```

Correct! 

But remember:

None

```
char → char *  
int → int *  
float → float *
```

EASY ANALOGY

Imagine a storage box .

If the box contains:

None

 Integer

we use an integer label.

If it contains:

None

 Character

we use a character label.

The pointer type tells C **what kind of value it should expect there**.

BIG MEMORY TRICK

None

```
int    → int *  
char   → char *  
float  → float *  
double → double *
```

★ TAKEAWAY

Pointer type tells C the type of data it points to.

SLIDE 5 — POINTERS AND DATA TYPES RECAP

WHAT DID WE LEARN?

1. INTEGER POINTER

None

```
int x = 10;  
int *p = &x;
```

int * → points to an integer

2. CHARACTER POINTER

None

```
char ch = 'A';  
char *p = &ch;
```

char * → points to a character

3. FLOAT POINTER

None

```
float x = 10.5;  
float *p = &x;
```

float * → points to a float

4. DEREFERENCE

None

```
*p
```

***p** → accesses the value

 **ONE BIG MEMORY TRICK!**

None

VARIABLE



DATA TYPE



POINTER TYPE

int → int *

char → char *

float → float *

And:

None

& → GET ADDRESS 

* → GET VALUE 

★ REMEMBER!

Pointer type matches the type of value it points to.

Question

1 What does **float *** mean?

- A) A pointer to an integer
- B) A pointer to a character
- C) A pointer to a float
- D) A normal float variable

 **Hint:** Look at the data type before *****.

Glossary:

Float → A number that can contain decimal values.

2 What is the output?

None

```
int x = 50;  
  
int *p = &x;  
  
printf("%d", *p);
```

- A) Address of `x`
- B) `50`
- C) `0`
- D) `p`

 **Hint:** `*p` accesses the value.

 **Glossary:**

Dereference → Access the value stored at the address held by a pointer.

3 Which pair is correctly matched?

- A) `int` → `char *`
- B) `char` → `float *`
- C) `float` → `float *`
- D) `int` → `double *`

 **Hint:** Match the same data type.

 **Glossary:**

Match → Correspond correctly with something.

4 What does the pointer store?

None

```
int x = 100;  
  
int *p = &x;
```

- A) The value `100`
- B) The address of `x`

C) The data type `int`

D) Nothing

 **Hint:** What does `&x` give us?

 **Glossary:**

Address → The memory location where a variable is stored.

FINAL CHALLENGE

Complete the code:

None

```
int x = 20;
```

```
---- p = &x;
```

```
printf("%d", ----);
```

 **Hint:**

- `x` is an `int`
- `&x` gives its address
- `*p` gives its value

 **Glossary:**

Pointer → A variable that stores the address of another variable.